

Mock OA — Problem Statement

4 levels, 800 points, 90 minutes

Prep materials — Anthropic Fellows OA

August 2026

Contents

Inference Engine Simulation — Assessment	1
Model	1
Level 1 — Sequential worker (200 pts)	1
Level 2 — Continuous batching with memory (200 pts)	2
Level 3 — Load balancer (200 pts)	2
Level 4 — Preemption & priorities (200 pts)	2
Notes	2

Inference Engine Simulation — Assessment

Implement the functions in `scheduler.py`. Do not modify `engine.py`. Run `python tests.py` to see your score (max **800**, 200 per level). Levels build on each other; partial credit is awarded per test.

Time budget: **90 minutes**. Start the clock before opening `scheduler.py`.

Model

Time advances in integer ticks. A request started at time s :

- spends $P = \text{ceil}(\text{prompt_tokens} / \text{prefill_rate})$ ticks in prefill
- then generates exactly one token per tick until `max_new_tokens` tokens have been produced
- its first output token exists at time $s + P + 1$
- it finishes at time $s + P + \text{max_new_tokens}$

Level 1 — Sequential worker (200 pts)

Implement `prefill_ticks(prompt_tokens, prefill_rate)` and `run_fifo(requests, config)`.

`run_fifo` simulates a single worker that serves one request at a time, in order of (`arrival`, `id`). A request cannot start before it arrives, and cannot start before the previous request finishes.

Example — `prefill_rate=4`, `requests (id=1, arrival=0, prompt=8, max_new=3), (id=2, arrival=1, prompt=4, max_new=2)`:

`run_fifo(...)` == {1: 5, 2: 8}

Level 2 — Continuous batching with memory (200 pts)

Implement `run_batched(requests, config)` for a single worker with a KV-cache budget of `config.memory_limit` tokens. Each request reserves `prompt_tokens + max_new_tokens` for its whole lifetime; the reservation is freed when it finishes.

Each tick $t = 0, 1, 2, \dots$ proceeds in three phases, strictly in this order:

1. **Free:** requests that finished at the end of the previous tick release their reservation.
2. **Admit:** waiting requests are considered in `(arrival, id)` order. Admit the head of the queue while it has arrived (`arrival <= t`) and its reservation fits. Stop at the first request that has arrived but does not fit (head-of-line blocking: later requests may NOT jump the queue).
3. **Work:** every running request advances one tick — `prefill_rate` prompt tokens of prefill, or one generated token if prefill is complete. A tick that completes prefill does not also generate a token.

Return a dict mapping `id -> (first_token_at, finished_at)`.

Level 3 — Load balancer (200 pts)

Implement `pick_worker(loads)` and `route(requests, config, num_workers)`.

`pick_worker(loads)` returns the index of the worker with the least load, breaking ties toward the lowest index.

`route` assigns requests to workers at arrival, in `(arrival, id)` order. A worker's load is the total reserved tokens of all requests assigned to it so far (assignments are permanent — no rebalancing). Each worker then runs its assigned requests independently, exactly as in Level 2.

Return `(assignment, results)` where `assignment` maps `id -> worker index` and `results` maps `id -> (first_token_at, finished_at)`.

Level 4 — Preemption & priorities (200 pts)

Implement `run_preemptive(requests, priorities, config)`. Single worker. `priorities` maps `id -> int`; **higher means more urgent**.

Changes from Level 2:

- Admission order is `(-priority, arrival, id)` — there is **no** head-of-line blocking; any arrived request may be considered.
- If the best candidate does not fit, **evict** running requests whose priority is *strictly lower* than the candidate's, until it fits or no such victim remains. Victim choice: lowest priority first, ties broken by **highest id**.
- An evicted request loses all progress (prefill must be redone) and returns to the queue with its original arrival. It may **not** be re-admitted on the same tick it was evicted.
- Repeat admission until no further request can be admitted this tick.

Return `(results, eviction_count)` where `results` is as in Level 2.

Notes

- Determinism matters: all ties are broken by lowest id / lowest index.
- The graders instantiate fresh `Request` objects per test; you may mutate the ones you're given.